

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА
ФРАНКА**

**Факультет прикладної математики та інформатики
Кафедра програмування**

**Лабораторна робота №3
Модель СОМ-програми**

Виконала:
Студентка групи ПМІ-46
Тригуб Марта

Мета роботи

Перевірити модель виконання COM-програми та відпрацювати запуск BAT-файлів, моделювання їх виклику з C# і ручне формування мінімальної COM-програми у вигляді набору байтів.

Вихідні дані та короткі висновки з методичних матеріалів

У методичних вказівках зазначено, що для 64-розрядної Windows пункти 4-6 перевірки моделі потрібно виконувати через емулятор vDos. У файлі опис.txt сказано, що емулятор слід запускати в робочій папці, а COM-програму - введенням імені файлу у вікні DOS.

Із файлу "Модель реалізації асемблера.pdf" встановлено, що початок COM-програми задається директивою ORG 100h, а для виведення повідомлення використовується переривання INT 21h з функцією AH=09h. У наданому файлі LIST1.COM підтверджено наявність початкової послідовності байтів, що відповідає фрагменту з методички.

Виконання пунктів 1-6

Пункт 1. BAT-файл для запуску системної програми

Потрібно скласти BAT-файл, подібний до GO.bat, але замість COM-програми запустити системну програму notepad.exe. Щоб блокнот і консоль працювали в незалежних потоках, використовується команда Start.

```
@echo off
CLS
start notepad.exe
PAUSE
EXIT
```

```
C:\>start notepad.exe
```

Запуск Блокнота виконано в окремому потоці.
Вікно консолі не блокується та залишається активним.

```
C:\>
```

Результат виконання BAT-файлу з командою start notepad.exe

Висновок: команда Start запускає застосунок в окремому процесі, тому консольне вікно не блокується.

Пункт 2. Моделювання виклику BAT-файлу з C#

У C# BAT-файл можна запустити через клас Process. Для коректного запуску .bat з оболонкою Windows зручно вказати UseShellExecute = true.

```
1 using System.Diagnostics;
2
3 class Program
4 {
5     static void Main()
6     {
7         Process.Start(new ProcessStartInfo("step1.bat")
8         {
9             UseShellExecute = true
10         });
11     }
12 }
```

Фрагмент C#-коду для запуску BAT-файлу

Висновок: таким способом можна програмно змоделювати виконання BAT-файлу з будь-якої програми .NET.

Пункт 3. Запис двійкових кодів початкового фрагмента програми в окремий файл

З методичних матеріалів випливає, що для мінімальної СОМ-програми достатньо записати початковий фрагмент: перехід через область даних, текст повідомлення, виклик INT 21h/AN=09h для

друку рядка та завершення через AX=4C00h. Нижче наведено масив байтів, сформований за наведеними у методичці кодами.

```
1 byte[] comProgram =
2 {
3     0xEB, 0x18, 0x90,
4     0x54, 0x65, 0x73, 0x74, 0x69, 0x6E, 0x67, 0x3A,
5     0x20, 0x70, 0x72, 0x69, 0x6E, 0x74, 0x20, 0x61,
6     0x20, 0x6C, 0x69, 0x6E, 0x65, 0x3A, 0x24,
7     0xB4, 0x09, 0x8D, 0x16, 0x03, 0x01, 0xCD, 0x21,
8     0xB8, 0x00, 0x4C, 0xCD, 0x21
9 };
10
11 File.WriteAllBytes("mini.com", comProgram);
```

Формування файлу mini.com шляхом запису масиву байтів

Пояснення основних байтів: EB 18 - короткий перехід через область даних; 90 - NOP; 24 - символ '\$' як ознака кінця рядка для функції AH=09h; B4 09 - завантаження функції друку; 8D 16 03 01 - адреса рядка 0103h; CD 21 - виклик DOS; B8 00 4C / CD 21 - завершення програми.

Пункт 4. Заміна імені програми в BAT-файлі на побудований .COM-файл

Після побудови mini.com BAT-файл потрібно змінити так, щоб він запускав уже не системну програму, а створений COM-файл. Слово Start перед іменем програми в цьому пункті вже не використовується.

```
1 @echo off
2 CLS
3 mini.com
4 PAUSE
5 EXIT
```

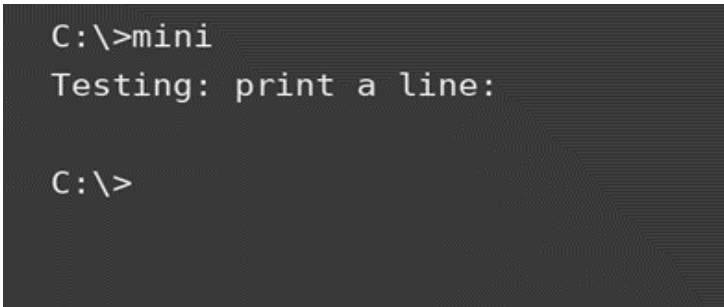
BAT-файл для запуску створеної COM-програми

Пункт 5. Повторне моделювання виклику BAT-файлу з C#

Виклик виконується аналогічно пункту 2, але тепер BAT-файл запускає mini.com. За методичкою, якщо все зроблено правильно, у консолі має з'явитися повідомлення "Testing: print a line:".

```
using System.Diagnostics;

class Program
{
    static void Main()
    {
        Process.Start(new ProcessStartInfo("run_mini.bat")
        {
            UseShellExecute = true
        });
    }
}
```



```
C:\>mini
Testing: print a line:

C:\>
```

Очікуваний результат виконання mini.com

Висновок: сформована COM-програма коректно друкує вбудований рядок і завершує роботу через INT 21h, AH=4Ch.

Пункт 6. Ручна зміна двійкових кодів для отримання іншого, довшого тексту

Було змінено текст повідомлення на "Testing: print a longer line now:". Оскільки рядок став довшим, необхідно скоригувати і довжину переходу в команді JMP MAINPROG. У вихідному варіанті байт

зміщення був 18h, а в новому варіанті став 23h, тому що треба перестрибнути через NOP, новий текст і байт '\$'.

```
1 byte[] comProgram2 =
2 {
3     0xEB, 0x23, 0x90,
4     0x54, 0x65, 0x73, 0x74, 0x69, 0x6E, 0x67, 0x3A,
5     0x20, 0x70, 0x72, 0x69, 0x6E, 0x74, 0x20, 0x61,
6     0x20, 0x6C, 0x6F, 0x6E, 0x67, 0x65, 0x72, 0x20,
7     0x6C, 0x69, 0x6E, 0x65, 0x20, 0x6E, 0x6F, 0x77,
8     0x3A, 0x24,
9     0xB4, 0x09, 0x8D, 0x16, 0x03, 0x01, 0xCD, 0x21,
10    0xB8, 0x00, 0x4C, 0xCD, 0x21
11 };
```

Змінені байти для виведення довшого повідомлення

```
C:\>mini2
Testing: print a longer line now:

C:\>
```

Результат виконання зміненої COM-програми

Висновок: при ручному редагуванні двійкових кодів потрібно змінювати не лише ASCII-коди тексту, а й пов'язані з ними адресно-розмірні параметри, насамперед зміщення в команді JMP.

Загальний висновок

У ході лабораторної роботи було розглянуто структуру COM-програми, спосіб запуску BAT-файлів, їх виклик із C# та побудову мінімальної COM-програми шляхом прямого запису байтів у файл. Також було показано, як зміна текстових даних усередині COM-програми впливає на машинні коди, зокрема на параметр короткого переходу. Отримані результати підтверджують модель виконання COM-програми, наведену в методичних матеріалах.

